

Máster Universitario IAASC

Lab 1 · Tema 1

Traza digital de una telco: eventos de red, clientes e incidencias

Caso transversal: operador de telecomunicaciones

Entorno: AWS Academy Learner Lab

Servicios: Amazon S3 · AWS Glue Data Catalog · Amazon Athena

Duración estimada: 120-150 minutos · Nivel: inicial

1. Idea general del laboratorio

Este laboratorio es la primera práctica de la asignatura y está directamente vinculado al Tema 1: La traza de datos como lenguaje del sistema complejo. Su objetivo no es entrenar un modelo, ni construir un cuadro de mando, ni implementar una arquitectura sofisticada. Su objetivo es mucho más fundamental: que el alumno aprenda a reconocer, capturar y consultar las distintas trazas digitales que produce un sistema complejo real —en este caso, un operador de telecomunicaciones— y a interpretar lo que esas trazas dicen y, sobre todo, lo que no dicen.

La operadora del caso transversal es un sistema sociotécnico que genera continuamente trazas heterogéneas: eventos de red, datos maestros de clientes, tickets técnicos, contactos con el call center, métricas de calidad y reclamaciones en lenguaje natural. Cada tipo de traza tiene un modelo de evento diferente, una granularidad temporal distinta y un nivel de estructura propio. El laboratorio guía al alumno para que cargue estas seis fuentes en Amazon S3, las registre en AWS Glue Data Catalog, las consulte con Amazon Athena y reflexione sobre lo que cada una aporta al observatorio del sistema.

Es importante subrayar lo que este laboratorio no hace, para no generar expectativas equivocadas. No construye modelos predictivos: eso corresponde a los Labs 3 y 4. No detecta anomalías: eso es el Lab 7. No aplica LLMs sobre el texto: eso es el Lab 8. Aquí trabajamos la capa más básica y, en gran medida, la más descuidada: la capa de la traza. Sin esta capa bien comprendida, todas las posteriores se construyen sobre suelo inestable.

Objetivos didácticos

Al finalizar el lab, el alumno debe ser capaz de:

1. Identificar los principales tipos de traza que produce una operadora de telecomunicaciones.
2. Cargar conjuntos de datos heterogéneos en Amazon S3 organizándolos por dominio y tipo.
3. Registrar tablas externas en AWS Glue Data Catalog para que Athena pueda consultarlas.
4. Escribir consultas SQL básicas en Athena sobre cada tipo de traza.
5. Caracterizar el modelo de evento de cada dataset: marca de tiempo, actor, acción y contexto.
6. Comparar la granularidad temporal y la cobertura de las distintas trazas.
7. Reconocer al menos tres limitaciones estructurales de la traza disponible y proponer cómo mitigarlas.
8. Producir un entregable breve que sirva como diagnóstico inicial del observatorio de la operadora.

Conexión con el Tema 1

El Tema 1 distingue, en su sección 1.3, ocho tipos principales de traza (logs, eventos de negocio, métricas, trazas distribuidas, datos de sensor, transacciones, texto no estructurado

y datos espaciales). De esos ocho tipos, este laboratorio trabaja con cinco directamente: logs (eventos de red), eventos de negocio (tickets, contactos), métricas (calidad de red), datos estructurados de cliente y texto no estructurado (reclamaciones). El laboratorio es, por tanto, una aplicación práctica de la tipología del tema.

El Tema 1 también introduce, en su sección 1.5, los límites e interpretaciones erróneas habituales de la traza. La parte final de este laboratorio retoma esa sección y obliga al alumno a reflexionar explícitamente sobre qué no aparece en la traza de la operadora, por qué no aparece y qué consecuencias tiene eso para la gestión.

2. Contexto del caso y servicios AWS

Caso de negocio

La operadora de telecomunicaciones de nuestro caso transversal opera servicios de fibra óptica, móvil y paquetes convergentes en seis regiones españolas: Madrid, Barcelona, Valencia, Sevilla, Bilbao y Zaragoza. Su dirección de operaciones ha planteado al equipo de datos una petición aparentemente sencilla: "queremos entender qué está pasando en nuestro servicio para tomar decisiones mejor informadas".

Esta petición, formulada así, es analíticamente inabordable. Antes de responderla, el equipo de datos debe hacer algo previo: inventariar y poner en consulta las trazas que el sistema produce. Sin ese inventario, cualquier respuesta posterior será una corazonada disfrazada de informe. Este laboratorio es, precisamente, ese paso previo: el alumno aterrizará seis fuentes de traza en Amazon S3 y las dejará consultables desde Amazon Athena para que cualquier pregunta posterior pueda apoyarse en evidencia digital concreta.

Servicios AWS utilizados

Servicio	Función en el laboratorio
Amazon S3	Almacenar los CSV de traza en su estado original, organizados por tipo.
AWS Glue Data Catalog	Registrar el esquema de cada dataset para que pueda consultarse como tabla SQL.
Amazon Athena	Ejecutar consultas SQL sobre los CSV directamente desde S3, sin servidor propio.

Tabla 1. Servicios AWS empleados en el Lab 1.

Amazon S3 actúa como almacén objeto: cada CSV se guarda como un objeto en un bucket, y permanece allí mientras dure el laboratorio. AWS Glue Data Catalog es un catálogo de metadatos: no almacena datos, almacena la descripción de los datos (qué columnas tiene cada tabla, de qué tipo, dónde están físicamente en S3). Amazon Athena es un motor de consulta SQL serverless que utiliza el catálogo de Glue para localizar e interpretar los datos en S3 y devolver los resultados. Esta separación entre almacenamiento, catálogo y motor es uno de los patrones más extendidos en arquitecturas analíticas modernas, conocido como "data lake con catálogo".

Datasets de partida

Se proporcionan seis ficheros CSV que representan tipos distintos de traza. La siguiente tabla describe cada uno:

Fichero	Tipo de traza	Filas	Granularidad
network_events.csv	Logs / eventos de red	350	Evento individual

Fichero	Tipo de traza	Filas	Granularidad
customers.csv	Maestro estructurado	500	Un registro por cliente
tickets.csv	Eventos de negocio + texto	420	Un registro por ticket
call_center_contacts.csv	Eventos de interacción	600	Un registro por contacto
network_metrics.csv	Métricas time-series	1.008	Una medida por hora y región (7 días)
complaints.csv	Texto no estructurado	120	Una reclamación libre

Tabla 2. Datasets de partida del laboratorio.

Nota. Los datos son sintéticos y han sido generados expresamente para fines docentes. No representan a ninguna operadora real, pero respetan la estructura y los patrones típicos del sector.

3. Preparación previa en AWS Academy

AWS Academy proporciona un entorno gestionado en el que los alumnos disponen de una cuenta AWS preconfigurada con permisos limitados y un crédito acotado. Es necesario tener presentes algunas particularidades de este entorno antes de empezar.

Particularidades del Learner Lab

- La sesión del laboratorio se inicia con el botón Start Lab y dura un tiempo limitado (habitualmente cuatro horas). Cuando se agota, los recursos persisten un tiempo, pero conviene completar el lab en una sola sesión continua.
- La región por defecto suele ser us-east-1 (Norte de Virginia). Conviene mantenerla y no cambiarla a menos que se indique expresamente.
- No es posible crear usuarios IAM ni roles personalizados. AWS Academy proporciona un rol predefinido llamado LabRole (a veces denominado voclabs) que ya tiene los permisos necesarios.
- Algunos servicios están bloqueados o restringidos. Para este lab, los tres servicios necesarios (S3, Glue y Athena) están disponibles sin restricciones relevantes.
- Es responsabilidad del alumno detener el lab (botón End Lab) al terminar, para liberar recursos y conservar crédito para sesiones posteriores.

Material necesario

- Acceso a AWS Academy con la cuenta del Máster.
- Los seis ficheros CSV proporcionados con el lab.
- Un editor de texto plano para anotar nombres de bucket, base de datos y consultas.
- Un navegador moderno (Chrome, Edge o Firefox actualizados).

Iniciar la sesión de AWS Academy

1. Accede a tu curso en AWS Academy a través del campus virtual.
2. Localiza el módulo de laboratorios y abre el Learner Lab correspondiente a esta asignatura.
3. Pulsa el botón Start Lab y espera a que el círculo del lado izquierdo cambie a verde. Este proceso puede tardar entre uno y tres minutos.
4. Cuando el círculo esté verde, pulsa el botón AWS en la parte superior de la página. Se abrirá la consola de AWS en una pestaña nueva.
5. Comprueba en la esquina superior derecha que la región seleccionada es us-east-1 (N. Virginia). Si no lo está, cámbiala antes de continuar.

Nota. Si la sesión se interrumpe, los datos en S3 y el catálogo de Glue persisten durante el periodo de inactividad permitido. Al retomar, basta con volver a pulsar Start Lab y entrar de nuevo a la consola.

Parte A. Configuración de Amazon S3

En esta parte vamos a crear el bucket que contendrá todos los CSV del laboratorio, organizados por tipo de traza. Es la pieza más importante: si los datos no están en S3, Athena no puede consultarlos.

Paso A.1. Crear el bucket

1. En la consola de AWS, en la barra de búsqueda superior, escribe S3 y abre el servicio.
2. Pulsa el botón naranja Create bucket.
3. Introduce un nombre de bucket único globalmente. Sugerencia de nomenclatura: unir-lab1-telco-traza-<tu-iniciales>-<año>. Por ejemplo: unir-lab1-telco-traza-jcp-2025.
4. Mantén la región us-east-1 (US East · N. Virginia).
5. Deja el resto de opciones con los valores por defecto. En particular, mantén activado el bloqueo de acceso público (Block all public access).
6. Pulsa Create bucket. El bucket aparecerá en la lista.

Nota. El nombre del bucket es único a nivel global de AWS, no de la cuenta. Si te dice que ya existe, prueba con otro sufijo (fecha, número aleatorio, etc.).

Paso A.2. Crear estructura de carpetas

Una buena práctica desde el primer día es separar los datos crudos, los resultados de las consultas y las posibles transformaciones. Aunque S3 no tiene carpetas reales (todo es un objeto con prefijo), la consola permite simularlas, y eso ayuda mucho a la legibilidad.

Entra en el bucket recién creado y crea las siguientes carpetas:

```
raw/  
  network_events/  
  customers/  
  tickets/  
  call_center_contacts/  
  network_metrics/  
  complaints/  
athena-results/
```

Para crear cada una, pulsa Create folder, escribe el nombre exactamente como aparece arriba (incluida la barra final que la consola añade automáticamente) y confirma. La estructura final será:

```
s3://unir-lab1-telco-traza-jcp-2025/raw/network_events/  
s3://unir-lab1-telco-traza-jcp-2025/raw/customers/  
s3://unir-lab1-telco-traza-jcp-2025/raw/tickets/  
s3://unir-lab1-telco-traza-jcp-2025/raw/call_center_contacts/  
s3://unir-lab1-telco-traza-jcp-2025/raw/network_metrics/  
s3://unir-lab1-telco-traza-jcp-2025/raw/complaints/  
s3://unir-lab1-telco-traza-jcp-2025/athena-results/
```

Nota. Sustituye el nombre del bucket por el tuyo en todas las rutas que aparezcan en el resto del documento.

Parte B. Subida de los datasets a S3

Cada CSV debe subirse a su carpeta correspondiente. Esto es importante porque, en la Parte D, vamos a registrar cada carpeta como una tabla independiente en Glue, y Athena leerá todos los objetos contenidos en ella como filas de esa tabla.

Paso B.1. Subir cada fichero

Para cada uno de los seis CSV repite el siguiente procedimiento:

1. Entra en la carpeta destino dentro del bucket (por ejemplo, raw/network_events/).
2. Pulsa Upload.
3. Selecciona Add files y elige el CSV correspondiente desde tu equipo.
4. Pulsa Upload en la parte inferior.
5. Espera al mensaje de confirmación Upload succeeded.

La correspondencia entre fichero y carpeta debe ser exactamente:

Fichero CSV	Carpeta destino en S3
network_events.csv	raw/network_events/
customers.csv	raw/customers/
tickets.csv	raw/tickets/
call_center_contacts.csv	raw/call_center_contacts/
network_metrics.csv	raw/network_metrics/
complaints.csv	raw/complaints/

Tabla 3. Correspondencia fichero / carpeta en S3.

Paso B.2. Verificación

Vuelve a la raíz del bucket y comprueba que la estructura es correcta. Una forma rápida de verificarlo es entrar en cada subcarpeta de raw y comprobar que contiene exactamente un fichero CSV. Si alguna carpeta está vacía o contiene un fichero erróneo, repite la subida.

Nota. No subas todos los CSV a la misma carpeta. Si lo haces, Athena los leerá como si fueran fragmentos de una sola tabla y dará errores de esquema, porque las columnas no coinciden entre datasets.

Parte C. Configuración inicial de Amazon Athena

Athena necesita dos cosas antes de poder consultar nada: saber dónde guardar los resultados de las consultas (un prefijo en S3) y disponer de una base de datos en el catálogo de Glue. Vamos a configurar ambas.

Paso C.1. Abrir Athena

1. En la barra de búsqueda de la consola de AWS, escribe Athena y abre el servicio.
2. Si es la primera vez que entras, te aparecerá un mensaje invitándote a configurar la ubicación de resultados. Pulsa Edit settings.
3. En el campo Query result location, introduce: s3://<tu-bucket>/athena-results/ (cambia el nombre del bucket por el tuyo).
4. Deja el resto de valores por defecto (encryption: none, etc.).
5. Pulsa Save.

Paso C.2. Verificar el catálogo

En el panel izquierdo del editor de consultas, comprueba que:

- Data source: AwsDataCatalog
- Database: (por ahora, default; lo cambiaremos enseguida).

Paso C.3. Crear la base de datos

En el editor de consultas, ejecuta:

```
CREATE DATABASE IF NOT EXISTS unir_telco_lab1;
```

Pulsa Run. Cuando termine, en el desplegable Database del panel izquierdo selecciona unir_telco_lab1. A partir de aquí, todas las tablas que creemos quedarán dentro de esta base de datos.

Nota. AWS Academy puede tardar unos segundos en refrescar el desplegable. Si no aparece, pulsa el icono de refrescar al lado del nombre de la base de datos.

Parte D. Registrar las tablas en Glue Data Catalog

En esta parte vamos a registrar cada CSV como una tabla externa. "Externa" significa que los datos no se mueven: permanecen en S3, y Glue solo guarda el esquema (qué columnas tiene, de qué tipo, dónde están). Athena utilizará ese esquema para interpretar el CSV cuando se ejecute una consulta.

Nota. Todas las sentencias siguientes deben ejecutarse con la base de datos `unir_telco_lab1` seleccionada en el editor de Athena. Sustituye el nombre del bucket por el tuyo en cada LOCATION.

Paso D.1. Tabla `network_events`

```
CREATE EXTERNAL TABLE IF NOT EXISTS network_events (  
    event_id          string,  
    event_timestamp   string,  
    node_id           string,  
    region            string,  
    event_type        string,  
    severity          string,  
    duration_seconds  int,  
    affected_customers int  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'  
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/network_events/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Nota. Hemos llamado a la columna `event_timestamp` en lugar de `timestamp` porque `timestamp` es palabra reservada en muchos dialectos SQL. Es una buena práctica evitar nombres reservados desde el primer día.

Paso D.2. Tabla `customers`

```
CREATE EXTERNAL TABLE IF NOT EXISTS customers (  
    customer_id      string,  
    region           string,  
    contract_type    string,  
    monthly_price     double,  
    tenure_months    int,  
    signup_date      string,  
    active           int  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'
```

```
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/customers/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Paso D.3. Tabla tickets

```
CREATE EXTERNAL TABLE IF NOT EXISTS tickets (  
    ticket_id          string,  
    event_timestamp    string,  
    customer_id        string,  
    category           string,  
    status             string,  
    priority           string,  
    resolution_hours   string,  
    description        string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'  
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/tickets/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Nota. `resolution_hours` está tipada como `string` porque `OpenCSVSerde` no maneja bien los valores vacíos en columnas numéricas. Más adelante haremos un cast a `double` cuando necesitemos operar con ella.

Paso D.4. Tabla call_center_contacts

```
CREATE EXTERNAL TABLE IF NOT EXISTS call_center_contacts (  
    contact_id          string,  
    event_timestamp     string,  
    customer_id         string,  
    channel             string,  
    reason              string,  
    duration_minutes     double,  
    resolved            int,  
    agent_id            string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'  
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/call_center_contacts/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Paso D.5. Tabla network_metrics

```
CREATE EXTERNAL TABLE IF NOT EXISTS network_metrics (  
    event_timestamp    string,  
    region             string,  
    avg_download_mbps double,  
    avg_upload_mbps   double,  
    avg_latency_ms     double,  
    packet_loss_pct    double,  
    active_users       int  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'  
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/network_metrics/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Paso D.6. Tabla complaints

```
CREATE EXTERNAL TABLE IF NOT EXISTS complaints (  
    complaint_id    string,  
    event_timestamp string,  
    customer_id     string,  
    region          string,  
    channel         string,  
    sentiment_label string,  
    text            string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    'separatorChar' = ',',  
    'quoteChar'     = '"',  
    'escapeChar'    = '\\'  
)  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/complaints/'  
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Paso D.7. Verificación rápida

Comprueba que las seis tablas se han creado con esta consulta:

```
SHOW TABLES IN unir_telco_lab1;
```

Y, para confirmar que los datos se leen correctamente, ejecuta para cada tabla:

```
SELECT COUNT(*) AS total FROM network_events;  
SELECT COUNT(*) AS total FROM customers;  
SELECT COUNT(*) AS total FROM tickets;  
SELECT COUNT(*) AS total FROM call_center_contacts;  
SELECT COUNT(*) AS total FROM network_metrics;
```

```
SELECT COUNT(*) AS total FROM complaints;
```

Los totales esperados son, aproximadamente, 350, 500, 420, 600, 1.008 y 120 filas respectivamente. Si alguno aparece como 0, lo más probable es que el CSV se haya subido a una carpeta equivocada o que la LOCATION de la tabla no apunte exactamente a la subcarpeta correspondiente.

Parte E. Exploración por tipo de traza

En esta parte realizaremos una primera consulta exploratoria sobre cada uno de los seis datasets. El objetivo no es responder preguntas de negocio: el objetivo es ver, con las propias manos, qué cuenta cada tipo de traza y qué granularidad ofrece.

Paso E.1. Logs: eventos de red

Empezamos por la traza más cercana al sistema físico: los eventos que la red de la operadora ha emitido durante el periodo observado.

```
SELECT event_type,
       severity,
       COUNT(*) AS total_eventos,
       ROUND(AVG(duration_seconds), 0) AS duracion_media_seg,
       SUM(affected_customers) AS total_afectados
FROM   network_events
GROUP BY event_type, severity
ORDER BY total_eventos DESC;
```

Preguntas de interpretación:

- ¿Qué tipo de evento es más frecuente? ¿Cuál es el más grave?
- ¿Por qué algunos eventos tienen duración 0 segundos? ¿Qué significa una duración 0 en un evento de tipo link_up o config_change?
- ¿Qué dimensiones tiene cada registro? Identifica explícitamente: marca de tiempo, actor (el nodo), acción (event_type) y contexto (region, severity, duration).

Paso E.2. Estructurado: maestro de clientes

La siguiente traza es de naturaleza completamente distinta: no es un evento, es un estado. Cada fila describe a un cliente en su forma actual.

```
SELECT region,
       contract_type,
       COUNT(*) AS total_clientes,
       ROUND(AVG(monthly_price), 2) AS precio_medio,
       ROUND(AVG(tenure_months), 1) AS antiguedad_media_meses,
       SUM(active) AS activos
FROM   customers
GROUP BY region, contract_type
ORDER BY region, total_clientes DESC;
```

Preguntas de interpretación:

- ¿Es esto realmente una "traza" en el sentido del Tema 1, o es algo distinto? Discute si un maestro de clientes es traza, estado, o ambas cosas.
- ¿Qué información del cliente NO aparece aquí? ¿Es esa ausencia accidental o deliberada?
- ¿Qué granularidad temporal tiene esta tabla? ¿Cuál es la diferencia con la granularidad de network_events?

Paso E.3. Eventos de negocio con texto: tickets

```
SELECT category,
       priority,
       status,
       COUNT(*) AS total
FROM   tickets
GROUP BY category, priority, status
ORDER BY total DESC
LIMIT 20;
```

Para ver una muestra del texto libre que viaja con cada ticket:

```
SELECT ticket_id, category, priority, description
FROM   tickets
WHERE  priority IN ('alta', 'critica')
ORDER BY event_timestamp DESC
LIMIT 10;
```

Preguntas de interpretación:

- ¿Qué porcentaje aproximado de los tickets son de alta o crítica prioridad?
- La columna description es texto libre. ¿Qué dimensiones del incidente se podrían extraer de ese texto que NO están en las columnas estructuradas?
- Observa el campo resolution_hours: ¿qué pasa con los tickets en estado abierto o en_proceso? ¿Cómo afecta esto a cualquier análisis de tiempo de resolución?

Paso E.4. Eventos de interacción: call center

```
SELECT channel,
       reason,
       COUNT(*) AS total_contactos,
       ROUND(AVG(duration_minutes), 1) AS duracion_media_min,
       SUM(resolved) AS resueltos
FROM   call_center_contacts
GROUP BY channel, reason
ORDER BY total_contactos DESC
LIMIT 20;
```

Preguntas de interpretación:

- ¿Qué canal es el más utilizado? ¿Es eso una propiedad del cliente, del sistema o de ambos?
- ¿Qué razón de contacto requiere mayor duración media? ¿Y cuál tiene menor tasa de resolución?
- Un mismo cliente puede aparecer varias veces en esta tabla. ¿Qué implicaciones tiene esto si quisiéramos cruzarla con la tabla customers? Anticipa cómo se haría ese cruce más adelante.

Paso E.5. Métricas time-series: calidad de red

La siguiente traza es radicalmente distinta: no es un evento sino una medida continua. Cada hora, en cada región, se ha tomado una medición de la red.


```
SELECT region,
       ROUND(AVG(avg_download_mbps), 1) AS download_medio,
       ROUND(AVG(avg_latency_ms), 1)   AS latencia_medio,
       ROUND(AVG(packet_loss_pct), 3)   AS perdida_medio,
       ROUND(MIN(avg_download_mbps), 1) AS download_minimo,
       ROUND(MAX(avg_latency_ms), 1)    AS latencia_maxima
FROM   network_metrics
GROUP BY region
ORDER BY download_medio DESC;
```

Para ver el patrón diario, segmentando por franja horaria:

```
SELECT region,
       CAST(substr(event_timestamp, 12, 2) AS int) AS hora,
       ROUND(AVG(avg_download_mbps), 1) AS download_medio,
       ROUND(AVG(avg_latency_ms), 1)   AS latencia_medio
FROM   network_metrics
GROUP BY region,
       CAST(substr(event_timestamp, 12, 2) AS int)
ORDER BY region, hora;
```

Preguntas de interpretación:

- ¿Se observa un patrón horario claro? ¿En qué franja la red se degrada más?
- Una métrica horaria, ¿es suficiente para detectar un microcorte de dos minutos? ¿Qué tipo de problema escapa a esta granularidad?
- ¿Qué diferencia hay, en términos de modelo de evento, entre `network_events` y `network_metrics`? ¿Cuándo conviene cada uno?

Paso E.6. Texto no estructurado: reclamaciones

```
SELECT sentiment_label,
       channel,
       COUNT(*) AS total
FROM   complaints
GROUP BY sentiment_label, channel
ORDER BY total DESC;
```

Para ver algunos ejemplos del texto libre:

```
SELECT region, channel, sentiment_label,
       substr(text, 1, 200) AS extracto
FROM   complaints
WHERE  sentiment_label = 'negativo'
ORDER BY event_timestamp DESC
LIMIT 10;
```

Preguntas de interpretación:

- Observa los textos: ¿qué temas se repiten? ¿Qué tipo de problema parece más recurrente?
- La etiqueta `sentiment_label` viene precalculada. ¿Confías en ella? ¿Qué problemas puede tener una etiqueta de sentimiento aplicada automáticamente sobre un texto libre?

- ¿Qué información hay en el texto que NO está en las columnas estructuradas y que podría ser muy relevante para gestión?

Parte F. El modelo de evento en la práctica

El Tema 1 introduce el modelo de evento como unidad mínima de la traza, con sus cuatro dimensiones: temporal, actor, acción y contexto. En esta parte vamos a aplicar ese modelo a los seis datasets que hemos cargado, completando una tabla comparativa.

Paso F.1. Identificar las dimensiones

Para cada dataset, completa mentalmente (y después por escrito, en el entregable) la siguiente tabla. Aquí te dejamos la versión de referencia que el profesor utilizará como solución orientativa:

Dataset	Marca de tiempo	Actor	Acción / valor	Contexto principal
network_events	event_timestamp	node_id	event_type + severity	region, duration, affected
customers	signup_date (estado actual)	customer_id	— (estado, no acción)	region, contract, price
tickets	event_timestamp	customer_id	category, status, description	priority, resolution_hours
call_center_contacts	event_timestamp	customer_id + agent_id	channel + reason	duration, resolved
network_metrics	event_timestamp	region (no individual)	download / latency / loss	active_users
complaints	event_timestamp	customer_id	text + sentiment	region, channel

Tabla 4. Modelo de evento aplicado a cada dataset del laboratorio.

Paso F.2. Reflexión sobre la unidad mínima

Esta tabla revela algo importante: no todos los datasets son eventos en el mismo sentido. customers es un estado, no un evento. network_metrics es una medida agregada por región y hora, no una observación individual. tickets y contactos son eventos con actor humano. network_events son eventos con actor técnico.

Esta heterogeneidad es la realidad de cualquier sistema complejo. Mezclar los seis datasets como si fueran "la traza" del sistema, sin distinguir su naturaleza, es el primer error analítico que conviene evitar. Cada tipo aporta una perspectiva distinta y exige un tratamiento distinto.

Parte G. Granularidad y cobertura

La sección 1.3 del Tema 1 introduce dos decisiones técnicas que condicionan toda la analítica posterior: la granularidad temporal y la granularidad de actor. Veamos cómo se concretan en nuestros datos.

Paso G.1. Comparar granularidad temporal

Ejecuta esta consulta para ver el rango temporal cubierto por cada dataset:

```
SELECT 'network_events' AS dataset,
       MIN(event_timestamp) AS desde,
       MAX(event_timestamp) AS hasta,
       COUNT(*)              AS filas
FROM   network_events
UNION ALL
SELECT 'tickets',
       MIN(event_timestamp), MAX(event_timestamp), COUNT(*)
FROM   tickets
UNION ALL
SELECT 'call_center_contacts',
       MIN(event_timestamp), MAX(event_timestamp), COUNT(*)
FROM   call_center_contacts
UNION ALL
SELECT 'network_metrics',
       MIN(event_timestamp), MAX(event_timestamp), COUNT(*)
FROM   network_metrics
UNION ALL
SELECT 'complaints',
       MIN(event_timestamp), MAX(event_timestamp), COUNT(*)
FROM   complaints;
```

Preguntas de interpretación:

- ¿Cubren todos los datasets el mismo periodo temporal? ¿Hay alguno con cobertura más estrecha?
- ¿Qué consecuencias tiene para el análisis que un dataset cubra solo una semana y otro cubra todo el mes?
- Si quisiéramos cruzar incidencias de red con tickets, ¿qué problema temporal podríamos encontrar?

Paso G.2. Comparar granularidad de actor

Cuenta los actores distintos en cada dataset:

```
SELECT 'nodos en network_events' AS metrica, COUNT(DISTINCT node_id) AS valor FROM
network_events
UNION ALL
SELECT 'clientes en customers', COUNT(DISTINCT customer_id) FROM customers
UNION ALL
SELECT 'clientes en tickets', COUNT(DISTINCT customer_id) FROM tickets
UNION ALL
```

```
SELECT 'clientes en contactos', COUNT(DISTINCT customer_id) FROM  
call_center_contacts  
UNION ALL  
SELECT 'clientes en complaints', COUNT(DISTINCT customer_id) FROM complaints  
UNION ALL  
SELECT 'regiones en network_metrics', COUNT(DISTINCT region) FROM network_metrics;
```

Preguntas de interpretación:

- Tenemos 500 clientes registrados. ¿Cuántos aparecen en tickets? ¿Cuántos en reclamaciones? ¿Qué dice eso sobre la cobertura de la traza?
- Los clientes que nunca abren un ticket ni reclaman, ¿son clientes "satisfechos" o son clientes "silenciosos"? ¿Puede la traza distinguir entre ambos?
- network_metrics está agregado por región, no por nodo individual. ¿Qué información se pierde con esa agregación? ¿Qué se gana?

Parte H. Primeros cruces entre trazas

Hasta ahora hemos consultado cada dataset por separado. La verdadera potencia de tener todo en Athena, sin embargo, está en poder cruzar las trazas. Aquí proponemos tres cruces básicos. Su objetivo sigue siendo exploratorio: ver qué se ilumina cuando las trazas conversan entre sí.

Paso H.1. Tickets enriquecidos con datos de cliente

```
SELECT t.category,
       c.region,
       c.contract_type,
       COUNT(*) AS total_tickets
FROM   tickets t
JOIN   customers c ON t.customer_id = c.customer_id
GROUP BY t.category, c.region, c.contract_type
ORDER BY total_tickets DESC
LIMIT 25;
```

Preguntas de interpretación:

- ¿Hay alguna combinación región × tipo de contrato que destaque en volumen de tickets?
- Eso, ¿significa peor servicio en esa región, mayor exposición (más clientes), o un sesgo de captura? ¿Cómo lo distinguirías?

Paso H.2. Severidad de eventos vs clientes afectados por región

```
SELECT region,
       severity,
       COUNT(*) AS eventos,
       SUM(affected_customers) AS total_afectados
FROM   network_events
GROUP BY region, severity
ORDER BY region, severity;
```

Preguntas de interpretación:

- ¿Qué región acumula más eventos críticos?
- ¿Coincide esa región con la peor calidad de red según la consulta E.5?
- Si coincide, ¿podemos decir que la calidad de red explica la peor experiencia? ¿O necesitaríamos más evidencia? Recuerda lo dicho en la sección 1.5 sobre correlación y causalidad.

Paso H.3. Volumen de contacto por canal y región

```
SELECT c.region,
       cc.channel,
       COUNT(*) AS contactos,
       ROUND(AVG(cc.duration_minutes), 1) AS duracion_media
FROM   call_center_contacts cc
JOIN   customers c ON cc.customer_id = c.customer_id
```

```
GROUP BY c.region, cc.channel  
ORDER BY c.region, contactos DESC;
```

Preguntas de interpretación:

- ¿Hay regiones con un perfil de canal claramente distinto al resto?
- ¿Es eso una propiedad demográfica, tecnológica o un sesgo de oferta (por ejemplo, presencia o ausencia de tienda física)?

Parte I. Reconocimiento de los límites de la traza

Antes de cerrar el laboratorio, dedica al menos quince minutos a este ejercicio. Es probablemente la parte más importante de toda la práctica, aunque no requiera ejecutar ni una sola consulta.

Paso I.1. Lo que la traza NO captura

Releyendo las consultas que has ejecutado y los resultados que has obtenido, identifica al menos cinco cosas relevantes para gestionar la operadora que NO aparecen en ningún dataset. Algunas pistas para empezar:

- La satisfacción real del cliente (no la inferida del sentimiento de reclamaciones).
- Los problemas resueltos informalmente, por ejemplo en redes sociales o por familiares de empleados, que nunca generaron ticket.
- La percepción interna del equipo técnico sobre la calidad del servicio.
- Información sobre clientes potenciales que decidieron no contratar.
- Acciones competitivas del mercado (ofertas de otros operadores, llegada de fibra a un barrio, etc.).

Para cada elemento que identifiques, anota: por qué no está, qué consecuencia tiene su ausencia, y si sería razonable, costoso o imposible incorporarlo.

Paso I.2. Sesgos posibles

Revisa el material del Tema 1 (sección 1.5) e intenta identificar al menos tres sesgos potenciales en la traza del laboratorio. Pistas:

- Sesgo de muestreo: ¿hay clientes que sistemáticamente no aparecen en ciertos datasets?
- Sesgo de supervivencia: ¿estamos observando solo a los clientes que aún siguen en cartera?
- Reactividad: ¿hay alguna métrica que pueda estar siendo "optimizada" por los propios actores del sistema?
- Etiquetado: ¿quién decidió que una reclamación es "negativa"? ¿Es siempre fiable esa etiqueta?

Paso I.3. Granularidad y decisiones

Reflexiona sobre la siguiente pregunta y respóndela en el entregable: ¿es la granularidad disponible (horaria para métricas, evento individual para incidencias, etc.) suficiente para las decisiones que la dirección quiere tomar? ¿Para cuáles sí y para cuáles no?

Parte J. Entregable del alumno

El alumno debe entregar un documento de entre cuatro y seis páginas con los siguientes apartados:

1. Resumen del sistema observado. Una página describiendo brevemente la operadora, sus regiones, sus servicios y los principales tipos de incidencia detectados.
2. Inventario de trazas. Tabla en la que, para cada dataset, se indiquen: tipo de traza, granularidad temporal, granularidad de actor, dimensiones del modelo de evento (timestamp, actor, acción, contexto) y observación destacada.
3. Tres consultas significativas. Para cada una: la consulta SQL, una captura del resultado, y una interpretación de tres a cinco líneas. Al menos una de las tres debe ser un cruce entre dos datasets.
4. Tres hallazgos descriptivos. Tres afirmaciones que el alumno pueda sostener sobre la operadora apoyándose en las consultas anteriores. Deben ser descripciones, no diagnósticos ni recomendaciones.
5. Tres preguntas que el alumno propone para los próximos labs. Preguntas que las trazas sugieren pero que este lab no puede responder por sí solo.
6. Sección de límites. Al menos cinco elementos relevantes que no están capturados en la traza, tres sesgos potenciales identificados, y una reflexión final sobre la suficiencia de la granularidad para las decisiones de la operadora.
7. Captura de la estructura de S3 y de la lista de tablas en Athena. Como evidencia de la ejecución técnica del laboratorio.

Criterios de evaluación

Criterio	Peso	Insuficiente	Aceptable	Excelente
Ejecución técnica (S3, Glue, Athena)	20%	Faltan tablas o datos	Las 6 tablas cargadas correctamente	Cargadas y verificadas con consultas de control
Inventario y modelo de evento	20%	Confunde tipos	Tipología correcta	Distingue evento / estado / métrica con justificación
Consultas e interpretación	20%	Solo copia consultas	Interpreta resultados	Cruza datasets y formula hipótesis con prudencia
Hallazgos descriptivos	15%	Mezclan descripción y causa	Descripciones claras	Descripciones precisas con reconocimiento de límites
Reconocimiento de límites	15%	No identifica límites	Identifica algunos	Distingue muestreo, supervivencia, reactividad y proxy

Criterio	Peso	Insuficiente	Aceptable	Excelente
Calidad de escritura y formato	10%	Confuso	Claro	Profesional, autocontenido, citas correctas

Tabla 5. Criterios y rúbrica de evaluación.

Parte K. Limpieza de recursos y cierre de sesión

AWS Academy tiene crédito limitado, y conviene dejar el entorno ordenado tras el laboratorio. La limpieza se hace en tres pasos.

Paso K.1. Eliminar las tablas y la base de datos en Athena

```
DROP TABLE IF EXISTS network_events;  
DROP TABLE IF EXISTS customers;  
DROP TABLE IF EXISTS tickets;  
DROP TABLE IF EXISTS call_center_contacts;  
DROP TABLE IF EXISTS network_metrics;  
DROP TABLE IF EXISTS complaints;  
DROP DATABASE IF EXISTS unir_telco_lab1;
```

Estas sentencias eliminan únicamente el catálogo (metadatos). Los CSV permanecen en S3 hasta que los borremos manualmente.

Paso K.2. Vaciar y eliminar el bucket S3

1. Vuelve a S3 y entra en el bucket creado en la Parte A.
2. Pulsa Empty para vaciarlo. Tendrás que escribir permanently delete en el cuadro de confirmación.
3. Una vez vacío, pulsa Delete para eliminar el bucket. Tendrás que escribir el nombre del bucket para confirmar.

Nota. Si planeas reutilizar los datos para el Lab 2, puedes saltarte el K.2 y conservar el bucket. En ese caso, anota su nombre.

Paso K.3. End Lab en AWS Academy

1. Vuelve a la pestaña del Learner Lab.
2. Pulsa el botón End Lab.
3. Confirma cuando se te pida.
4. Verifica que el círculo del lado izquierdo cambia de verde a rojo.

Cierre didáctico

Has completado el primer laboratorio de la asignatura. No has entrenado ningún modelo, no has detectado ninguna anomalía y no has predicho ningún abandono. Lo que has hecho es algo más fundamental: has aterrizado en un entorno cloud real las seis trazas principales de una operadora de telecomunicaciones, has visto con tus propios ojos qué tipo de información aporta cada una, has practicado la lectura del modelo de evento subyacente y has reconocido los límites estructurales de ese observatorio.

Todo lo que la asignatura desarrolla después —preparación de datos, modelos predictivos, análisis de redes, modelos de lenguaje, mapas causales, auditoría de sesgos— se construye sobre esta capa. Sin un buen inventario de la traza, sin una buena comprensión de qué cuenta

cada fuente y qué calla, las técnicas posteriores se aplican a ciegas y producen confianza injustificada. El alumno que ha entendido este lab tiene una base sólida para todo lo que viene.

Referencias y lecturas complementarias

- Amazon Web Services. (2024). Amazon Athena User Guide. <https://docs.aws.amazon.com/athena/>
- Amazon Web Services. (2024). AWS Glue Data Catalog Developer Guide. <https://docs.aws.amazon.com/glue/>
- Amazon Web Services. (2024). Amazon S3 User Guide. <https://docs.aws.amazon.com/AmazonS3/>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.
- Kitchin, R. (2014). The data revolution: Big data, open data, data infrastructures and their consequences. SAGE Publications.
- Sridharan, C. (2018). Distributed systems observability: A guide to building robust systems. O'Reilly Media.